

Продукт

Как создать фабрику облачного программного обеспечения — проверка использования компьютера

Зак Ллойд 3 августа 2026 года



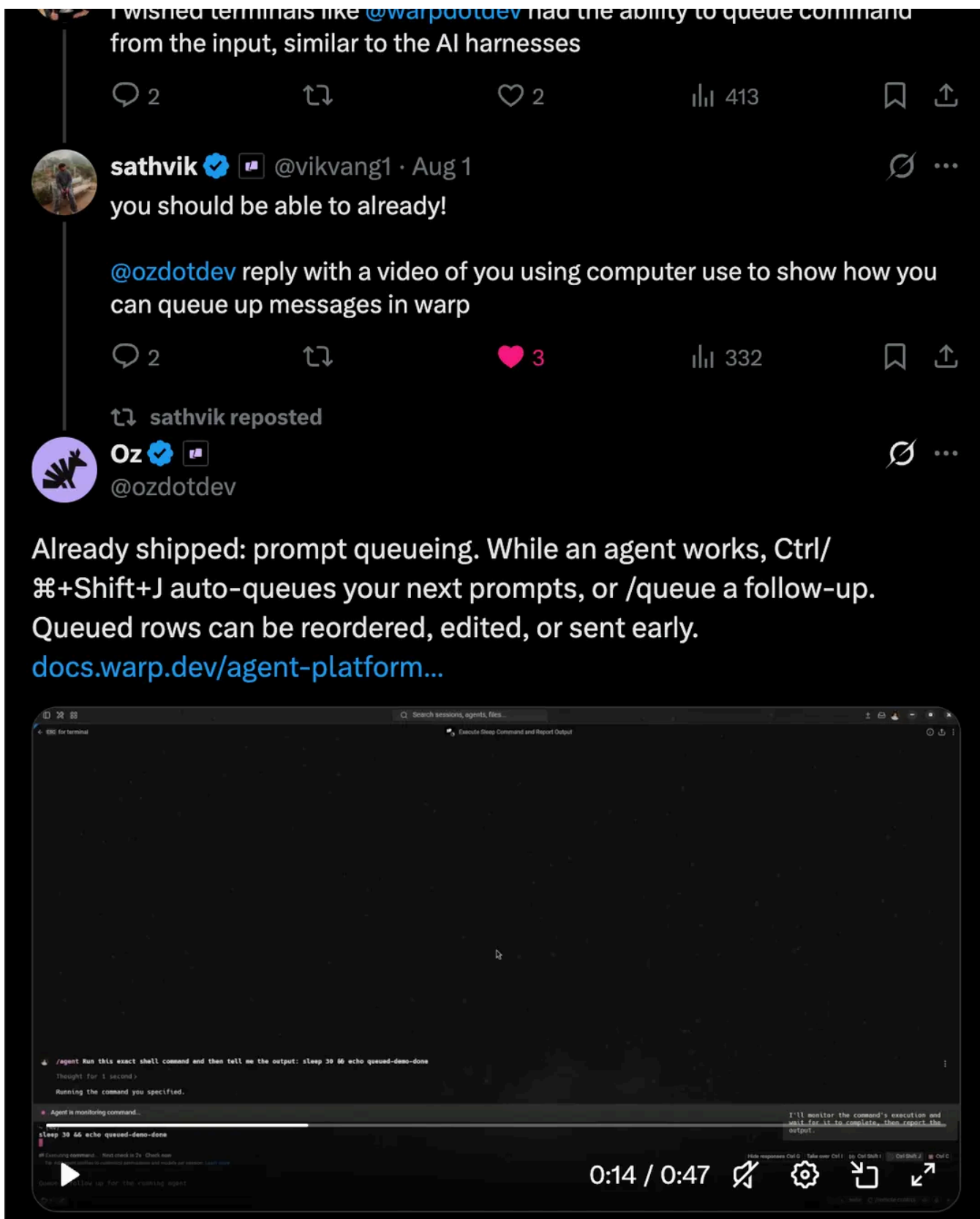
Это четвертый пост в моей серии о том, как создать фабрику облачного программного обеспечения.

В этом посте я расскажу, как добавить в работу ваших агентов использование компьютера и браузера для воспроизведения проблем и проверки исправлений и новых функций.

Предыдущие посты:

- 1 . Создайте агента для сортировки задач и агента-исполнителя
- 2 . Добавьте разработку на основе спецификаций с помощью агента спецификаций
- 3 . Добавьте самосовершенствующийся агент для проверки кода

Для тех, кто не знаком с использованием компьютера и браузера, поясним, что это инструмент, который позволяет агентам управлять запущенным приложением напрямую с помощью щелчков мыши и нажатий клавиш. У большинства крупных поставщиков моделей есть модели, поддерживающие эту функцию, и они становятся все более доступными в различных средах разработки, включая Warp.



Мы внедрили компьютерные технологии во все части нашего агентного стека, включая запросы на добавление функций в Twitter ↗

Ценность использования компьютера на вашей фабрике облачного программного обеспечения заключается не столько в нем самом как в отдельном агенте, сколько в

возможностях, которые он предоставляет другим агентам в производственном процессе. В частности, использование компьютера ценно:

1. На этапе триажа ↗ для воспроизведения ошибок перед попыткой их исправить
2. На этапе внедрения ↗ для проверки исправлений и соответствия новых функций спецификациям
3. На этапе проверки ↗ для подтверждения того, что код действительно соответствует ожидаемому поведению

Верификация особенно ценна как способ снизить нагрузку при проверке кода. Если вы видите видео, на котором функция работает, вы с большей вероятностью будете доверять лежащему в ее основе коду (хотя вам все равно нужно будет его проверить, чтобы убедиться, что архитектура хорошая, код чистый и безопасный и т. д.). Особенно в случае с чисто визуальными изменениями, которые сопряжены с низким риском, наличие «доказательства» того, что функция работает с точки зрения пользователя, ценно и экономит время.

Еще одно, менее очевидное преимущество использования компьютера заключается в том, что оно позволяет агенту создать цикл, в рамках которого он может самостоятельно отлаживать изменения. Это особенно хорошо работает при разработке на основе спецификаций, когда у вас есть подробный `PRODUCT.md` ↗, который агент пытается реализовать. На каждом этапе реализации агент может использовать компьютер, чтобы проверить, насколько реализация соответствует спецификации, и продолжать работу до тех пор, пока результат не будет удовлетворительным. Здесь важно следить за расходами, так как это может быть дорого, но повышает вероятность успеха.

Как и в предыдущих публикациях, вы можете использовать эту реализацию в своих репозиториях, следуя демонстрации с открытым исходным кодом ↗. Существует навык, который вы можете установить ↗, а затем использовать в своем любимом агенте для написания кода, чтобы напрямую настроить все в своей репозитории:

```
npx skills add warpdotdev-demos/cloud-factory-demo --skill oz-cloud-factory-demo
```

Добавьте проверку использования компьютера в работу своих агентов

При работе с компьютером мы создаем новый навык под названием `verify-behavior` ↗, который определяет, как использовать компьютер и браузер.

SKILL.md

Verify behavior

Prove or disprove visible product behavior for **bugs and greenfield features**. Other agents (triage, implementation, review) invoke this instead of driving the UI themselves.

Default evidence: **video** of the critical path, plus keyframe screenshots. Screenshots alone only when a single frame is clearer or video is unavailable.

Modes

- **reproduce** — show whether a reported bug still happens on baseline (usually default branch). Used mainly from triage.
- **verify** — show whether the implemented change matches expected behavior. Covers **features and fixes**. Used from implementation/review.

Infer mode if unnamed: issue-only → **reproduce** ; implementation/PR branch → **verify** .

The verify behavior skill ↗

The key aspects of the skill are:

- When to use computer use (desktop and mobile-native) vs. browser use (webapps)
- What to capture: video preferably, but screenshots are fine
- The two modes to use it in:
 - **reproduce**: try to confirm that a reported bug occurs
 - **verify**: verify a new behavior

In order for this skill to work, it needs to be run through a harness that supports computer and browser use. Since we are building a cloud software factory, this should be a harness that supports cloud agents. As with prior posts, we use Warp's cloud agent platform, but there are others that support it as well.

At this point, you can run the `verify-behavior` ↗ skill directly, but it's actually more useful to hook it into the other agents in your factory as a capability they can take advantage of. To

do this, we will update the existing Triage ↗, Review ↗ and Implementation ↗ skills to encourage them to use the verify-behavior skill when it will help them do their jobs.

For instance, we can update the Triage skill ↗ to try to reproduce bugs:

```
64 + ### 4. Optionally reproduce visible bugs with verify-behavior
65 +
66 + When the issue is a UI, browser, desktop, rendering, layout, or other interactive bug, and visual reproduction would
    materially improve the readiness decision, use the factory verification skill:
67 +
68 + 1. If `.agents/skills/verify-behavior/SKILL.md` exists, read it and follow its parent workflow in `reproduce` mode.
69 + 2. Launch the verification work as an Oz cloud computer-use **subagent** (do not drive the GUI yourself unless the
    user explicitly asks).
70 + 3. Prefer video evidence of the repro path; accept screenshots when they are clearer or video is unavailable.
71 + 4. Fold the reproduction status into your rationale. Confirmed repro strengthens `Ready to implement` or `Ready to
    spec` when the rest of the rubric fits. Failed or blocked repro often supports `Needs info` when steps or environment
    details are missing.
72 +
73 + Skip this step for non-visual issues, when the skill file is missing, or when computer-use verification is clearly
    unnecessary overhead for a trivial, already well-specified bug. Never block forever on verification: if the subagent
    is blocked, record that and continue with the best evidence-based state.
74 +
```

And update the Implementation skill ↗ to verify new features:

```
128 + ### 9. Verify visible behavior with verify-behavior
129 +
130 + When the change affects UI, browser, desktop, or other interactive behavior, and
    `.agents/skills/verify-behavior/SKILL.md` is present:
131 +
132 + 1. Read that skill and follow its parent workflow in `verify` mode. This covers greenfield
    features and bug fixes alike.
133 + 2. If `PRODUCT.md` exists for the issue, pass its path and treat it as the source of user
    stories and acceptance criteria verification must exercise.
134 + 3. For multi-story features, expect `verify-behavior` to **fan out parallel computer-use (or
    isolated browser) subagents per key user story** via orchestration, then aggregate results.
    Do not require a single serial walkthrough when stories are independent.
135 + 4. Launch verification against the implementation branch (or the commit about to be pushed).
136 + 5. Prefer **video** of each critical path; add screenshots for keyframes or static visual
    checks.
137 + 6. If verification fails because of your change, fix the implementation and re-run
    verification when practical before opening the PR.
138 + 7. If verification is blocked (missing display, secrets, flaky environment), record the
    blocker and continue with automated tests plus an explicit verification gap in the PR.
139 +
140 + Skip this step for non-visual changes, when the skill is missing, or when the repository has
    no runnable UI in the available environment. Do not claim end-to-end behavioral verification
    unless verify-behavior ran or you captured equivalent evidence.
141 +
```


The update to the implementation skill to verify behaviors

Note that the verify-behavior skill uses cloud subagents. You can also do computer use locally, but it tends to be a worse experience unless the platform you're using can test in the background (otherwise it steals focus and prevents you from doing anything while it runs). Warp and Codex do support this, but it still tends to be best in my experience to do computer use on a cloud machine where there's no chance of the agent doing anything weird with the apps running on your machine.

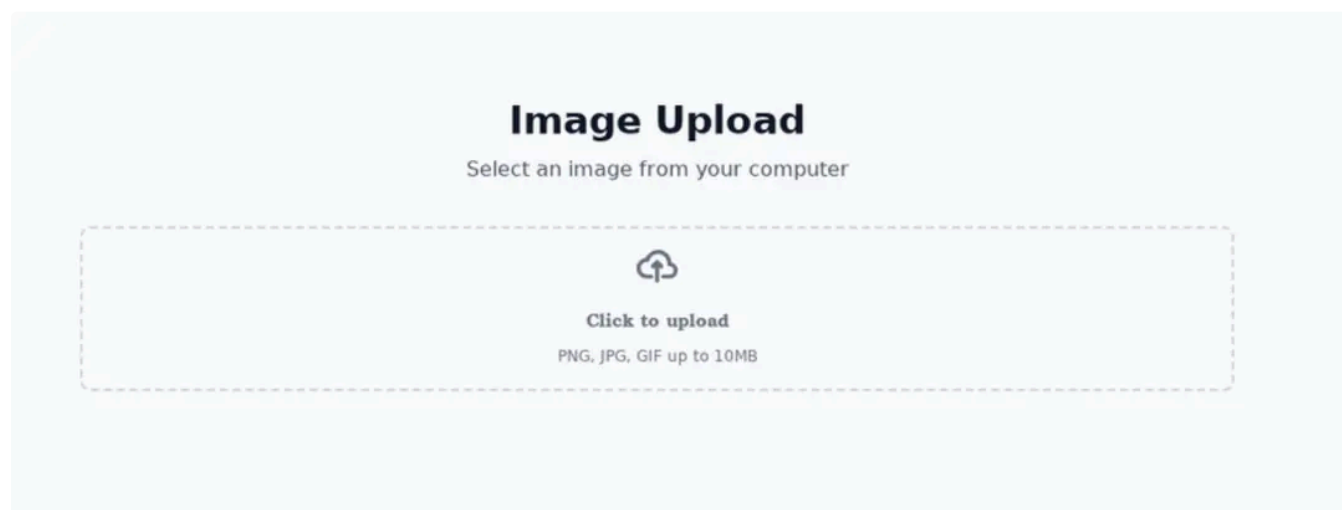
For complex or new behaviors, we ask our orchestrator to fan-out to verify all of the user stories independently and in-parallel (your harness needs to support multi-agent orchestration for this to work). Computer use is typically single-threaded, so if you want better throughput, it's best to fanout cloud agents across machines. This can get expensive, but it's very cool, and reduces latency a ton.\

Computer use in action

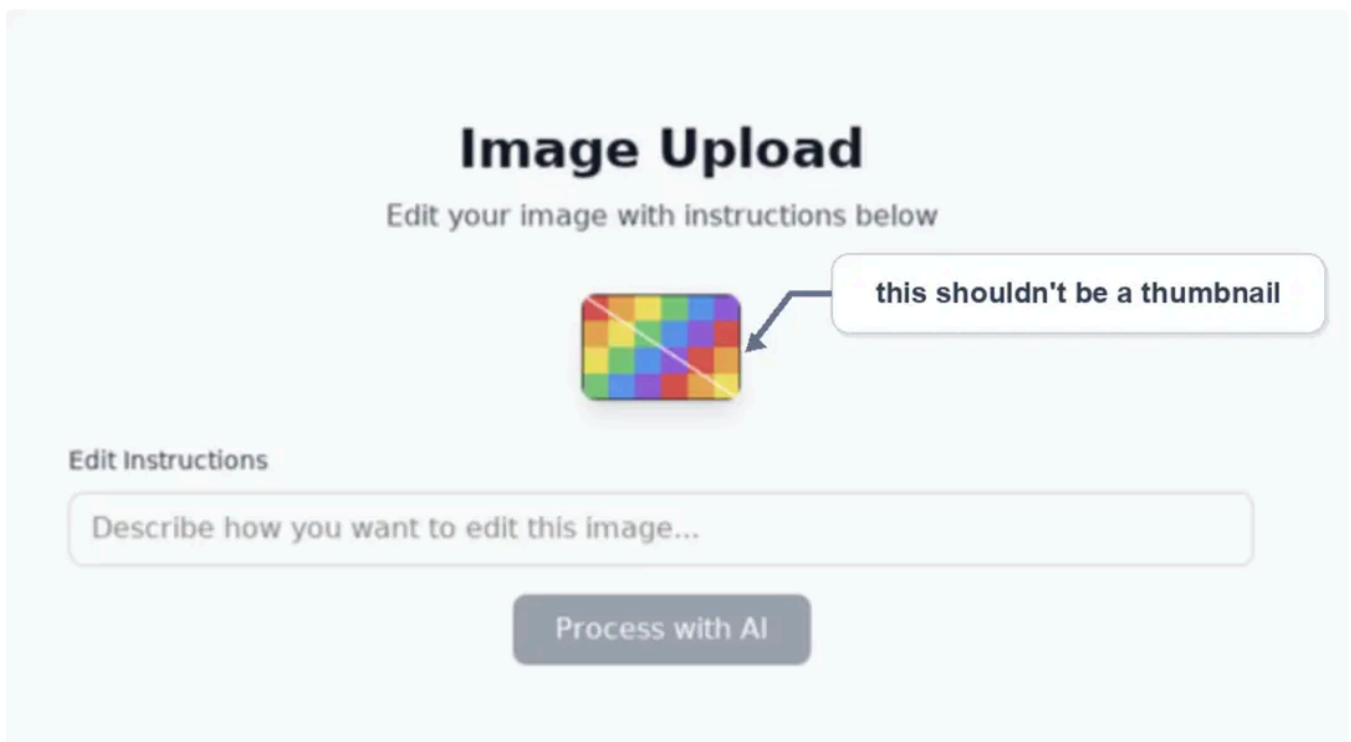
I have a [demo repo ↗](#) that I've used in a few of these posts that implements a simple agentic image editor using Nano Banana.

As a first user story, I'm going to show how you can use computer use to verify an issue. In this case, there's a bug in the demo app: when an image is uploaded, its preview renders at thumbnail size rather than gallery size. The GitHub issue where this was reported is [here ↗](#).

The agent here was able to run and produce screenshots of the broken behavior:



The zero state for reproducing the issue



A screenshot showing the incorrect thumbnail

This is a trivial example, but for folks dealing with bug reports, having images automatically generated to verify the reproduction is a big unlock. You can see the raw cloud agent run that generated these images [here](#) if you're curious what the agent trace looks like.

Moving on to a second use case, I asked an agent to implement a new feature in this repo end-to-end and verify it using computer use. The feature is described in this [issue](#) and is a relatively straightforward UI change, adding “clear” and “replace” controls to the image editor. There are clear acceptance criteria in the issue for the verifier to check against:

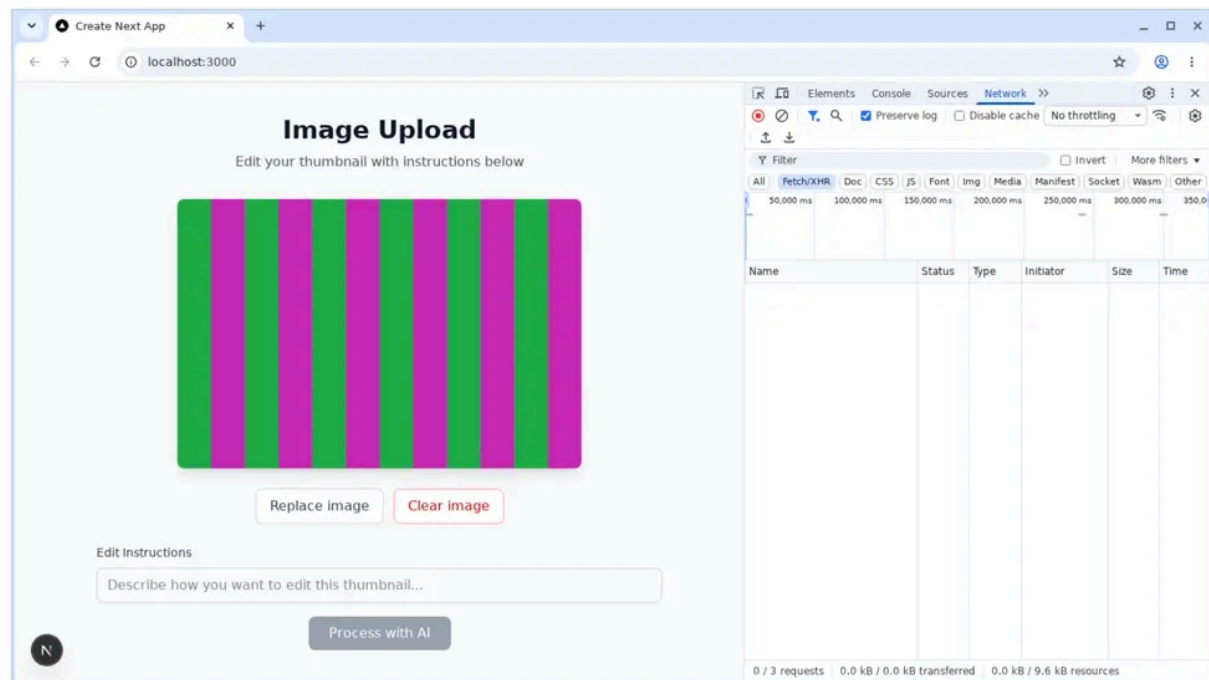
Acceptance criteria

- ☐ A visible `Replace image` control appears with a selected image ...
- ☐ Replace accepts image files and updates both the preview and selected file ...
- ☐ A visible `Clear image` control appears with a selected image ...
- ☐ Clear returns to the empty upload area without reloading the page ...
- ☐ Clear resets the selected image/file, instructions, status/error text, response text, and image history ...
- ☐ After Clear, uploading another image still works ...
- ☐ Both controls work by pointer and keyboard ...
- ☐ Neither action calls `/api/process-image` ...
- ☐ No new dependencies and no related console errors ...

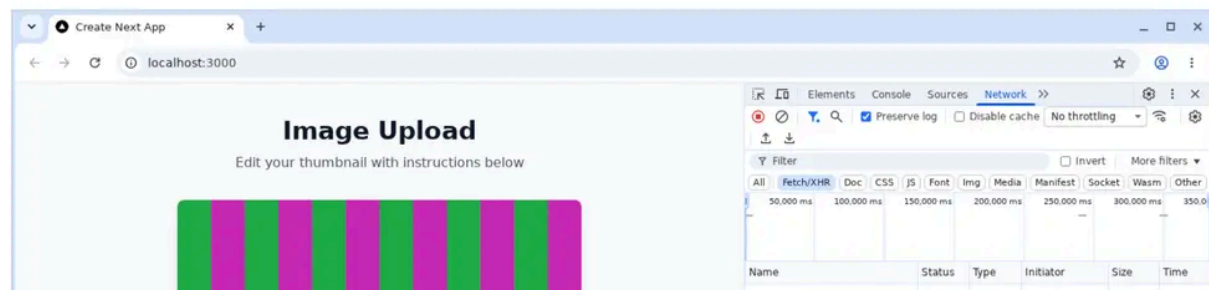
Acceptance criteria for `verify-behavior` to check

The PR [itself](#) has screenshots showing the controls, and a link to a video proving they work.

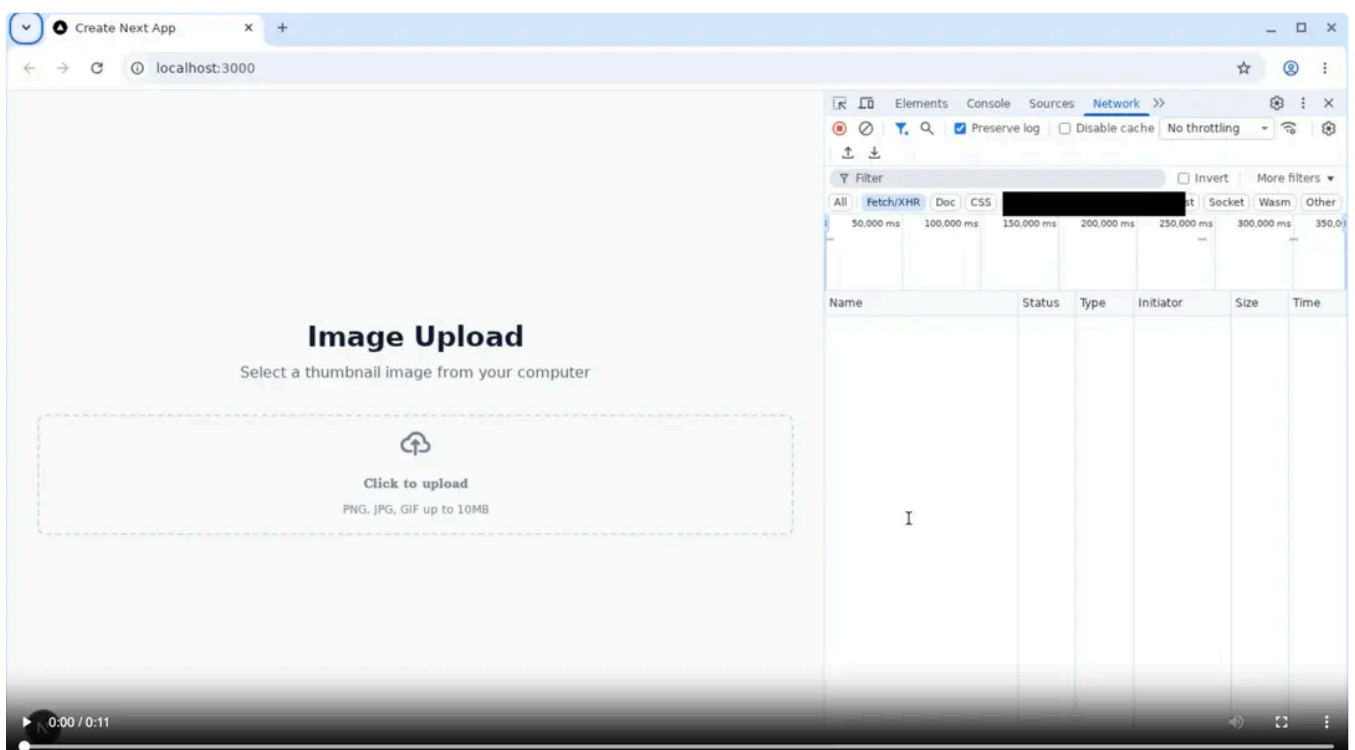
Computer-use screenshots



Selected-image state after upload — the `Replace image` / `Clear image` controls are present only while an image is selected.



Here's a video the agent produced showing the feature in action end-to-end:



It really is that simple – once you have the scaffolding set up you can use computer use to help lift the burden on PR review. In fact, for low-risk UI changes, you might decide that watching these videos is enough and you don't need to look at the code at all.

To recap, we have created agents that:

1. Triage new issues ↗, optionally sending them to be implemented or spec'd
2. Spec issues ↗ from a product and technical perspective
3. Implement the actual code ↗
4. Review the code ↗ and improve review over time with an observer loop

And now we've added a verification capability that all of the other agents can use to make sure the implementation is correct and allow a human reviewer to see a record of how the feature works.

These posts show how, with a simple set of primitives, you can start to build your own factory and truly automate more of your mundane work. The primitives are just:

- A cloud agent platform like Warp that supports computer-use
- A workflow tool like GitHub Actions
- A set of agent skills that you and an agent can tune

My next post in this series will show how you can expand this factory with agents that monitor features and fixes after they are released and feed that monitoring back into the Triage phase, completing a basic factory loop.

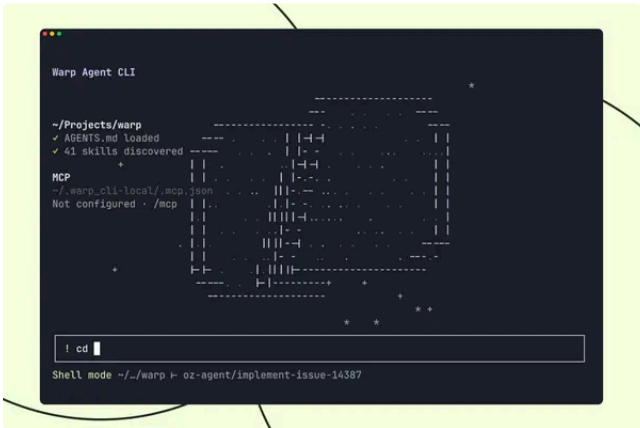
Start your software factory

Book a demo and we'll walk you through the workflows that map to your stack.

What's your work email?

Get Started

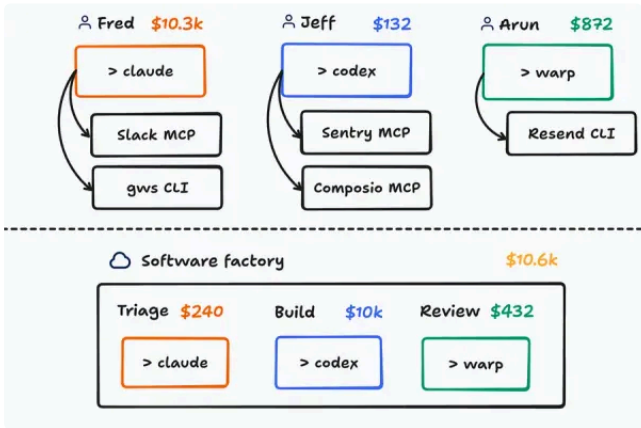
Related articles



AUG 4, 2026 · 6 MIN

Introducing the Warp Agent CLI: a CLI coding agent that does what others can't

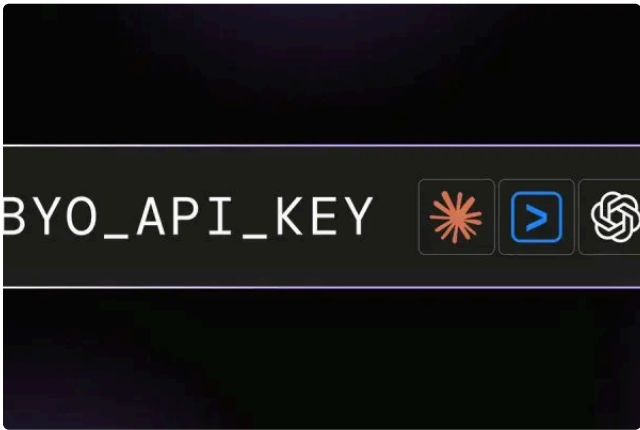
The Warp Agent is now available as a standalone CLI that you can use anywhere – Ghostty, iTerm 2, VS Code, built-in Windows and Mac Terminals –...



JUL 18, 2026 · 4 MIN

Get agents off your machine

Even though we are living in 2026, it feels like folks have forgotten the lessons of 2006: software belongs in the cloud, not on individuals' desktops.



MAY 20, 2026 · 6 MIN

Bring your own inference to Warp

Today we're releasing one of the most requested updates from the Warp community: more control over inference.

Get Warp today

Get early access to unreleased and experimental features with Warp Preview



Mac

macOS

Version 10.14+

```
$ brew install --cask warp
```

📄



Linux

.deb

Debian, Ubuntu

.rpm

Red Hat, Fedora, SUSE

.deb	x64	ARM64
.rpm	x64	ARM64
.tar.zst Arch Linux	x64	ARM64
ApplImage	x64	ARM64



Windows

.exe

Windows 11/10 x64

.exe

Windows 11/10 ARM64

```
$ winget install Warp.Warp
```



.exe

x64

ARM64

PRODUCT

[Download](#)

[Pricing](#)

[Enterprise](#)

[Changelog](#)

[Docs](#)

[Open Source](#) ↗

[Status](#)

LEGAL

[Privacy Policy](#)

[Terms of Service](#)

[Security](#)

[Cookie Notice](#)

[Data Processing Addendum](#)

[Enterprise License](#)

RESOURCES

[Blog](#)

[FAQ](#)

[Community](#)

[Support](#)

[Do Things with Warp](#)

[Terminus](#)

CONNECT

[X](#) ↗

[GitHub](#) ↗

[YouTube](#) ↗

[Slack](#)

[LinkedIn](#) ↗

[Product Hunt 4.8★ \(77\)](#) ↗

COMPANY

[About](#)

[How We Work](#)

[Careers](#)

[Press](#)

[Contact](#)

